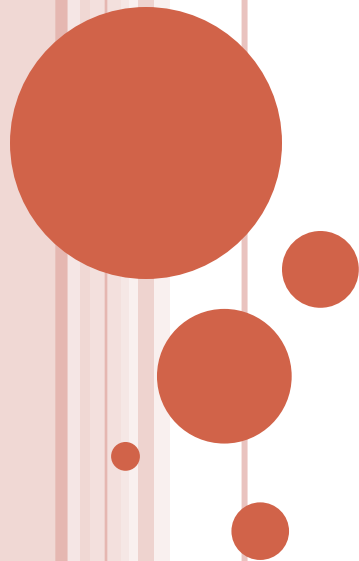


SOFTWARE ENGINEERING

SUBJECT CODE: MCAXX11509

UNIT 2 : SOFTWARE DESIGN



UNIT OUTLINE

- Importance of Software design
- Component of UML Diagrams..
 - Class Diagram
 - Object Diagram
 - Use case Diagram
 - Activity Diagram
- Case study on...
 - Library Management System
 - Ticket Reservation System
 - Hospital Management System
 - Online Customer Service System



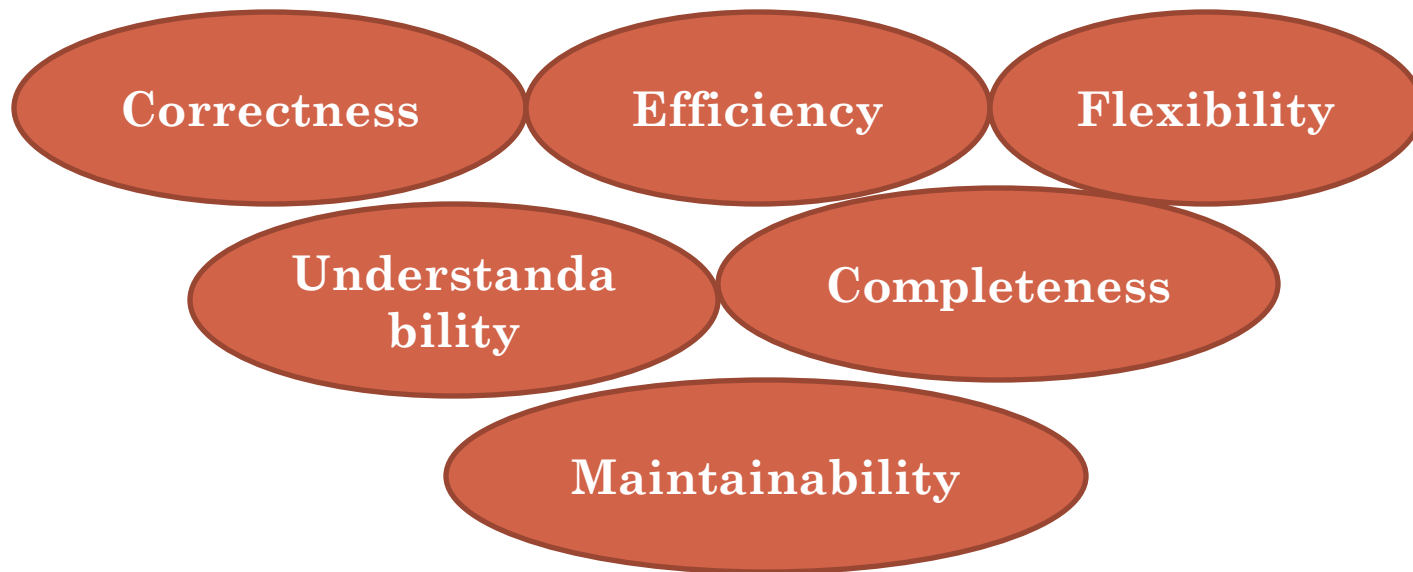
Importance of Software design

- Software design is a **mechanism to transform user requirements into some suitable form**, which **helps the programmer in software coding and implementation**.
- It deals with **representing the client's requirement, as described in SRS** (Software Requirement Specification) document, into a form, i.e., easily implementable using programming language.
- In software design, we consider the system to be a set of **components or modules with clearly defined behaviors & boundaries**.



➤ Objectives of Software Design

○ Following are the purposes of Software design:



1. Correctness:

- Software design **should be correct as per requirement.**
- A good design should be correct i.e., it should **correctly implement all the functionalities** of the system.

2. Efficiency:

- Resources should be used **efficiently by the program.**
- A **good software design** should address the **resources, time, and cost optimization issues.**

3. Flexibility:

- **Able to modify on changing needs.**
- A good software design should have the **ability to adapt and accommodate changes easily.**
- It includes designing the software in a way, **that allows for modifications, enhancements, and scalability without requiring significant rework or causing major disruptions** to the existing functionality.



4. Understandability:

- A good design should be **easily understandable**, it should be modular, and all the **modules are arranged in layers**.

5. Completeness:

- The design should have **all the components like data structures, modules, external interfaces**, etc.

6. Maintainability:

- A good software design aims **to create a system that is easy to understand, modify, and maintain over time**.
 - This involves using **well-structured design**
 - Maintainability in Software and design also enables **developers to fix bugs, enhance features, and adapt the software to changing requirements** without excessive effort or introducing new issues.
- 

UML(UNIFIED MODELING LANGUAGE)

1. What is UML?

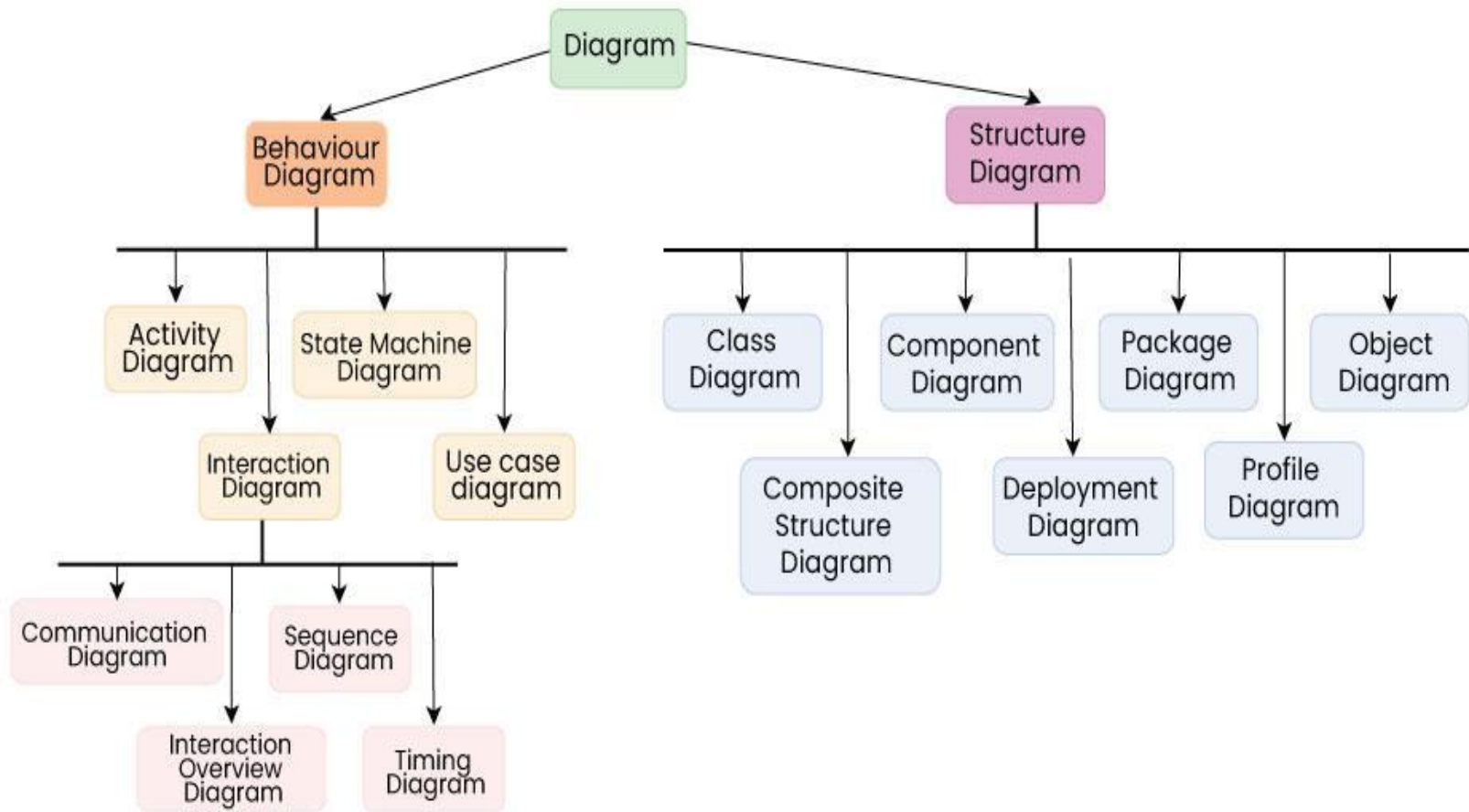
- Unified Modeling Language (UML) is a standardized visual modeling language that is a **versatile, flexible, and user-friendly method for visualizing a system's design.**
- We use UML diagrams to **show the behavior and structure of a system.**
- UML **helps software engineers and system architects with modeling, design, and analysis.**



➤ Why do we need UML?

- We need UML (Unified Modeling Language) to **visually represent and communicate complex system designs**, facilitating **better understanding and collaboration among stakeholders**.
- **Businessmen do not understand code**. So UML becomes essential to communicate with non-programmers about essential **requirements, functionalities, and processes of the system**.
- **A lot of time is saved** down the line when **teams can visualize processes, user interactions, and the static structure of the system**.





1.UML Class Diagram

- The class diagram depicts a **static view of an application**.
- It represents the **types of objects residing in the system and the relationships between them**.
- A class consists of **its objects**, and also it may **inherit from other classes**.
- It **shows the attributes, classes, functions, and relationships to give an overview of the software system**.
- It constitutes class names, attributes, and functions in a separate compartment that helps in software development.



➤ Benefits of Class Diagrams

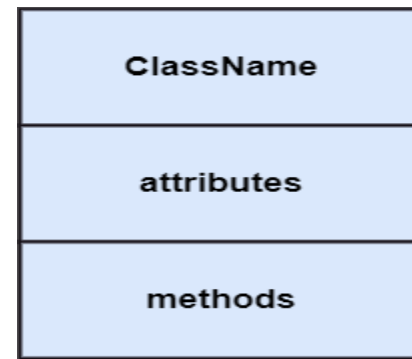
- It can represent the **object model for complex systems**.
- It **reduces the maintenance time** by providing an overview of how an application is structured before coding.
- It provides a **general schematic of an application** for better understanding.
- It is helpful for **the stakeholders and the developers**.



➤ Key components of a Class Diagram

1. Upper Section:

- The upper section **include the name of the class**.
- A class is a representation of **similar objects** that shares the same relationships, attributes, operations, and semantics.
- Some of the following rules that should be taken into account while representing a class are given below:
 - **Capitalize the initial letter** of the class name.
 - Place the **class name in the center of the upper section**.
 - A class name must be **written in bold format**.
 - The **name of the abstract class** should be written in **italics format**.



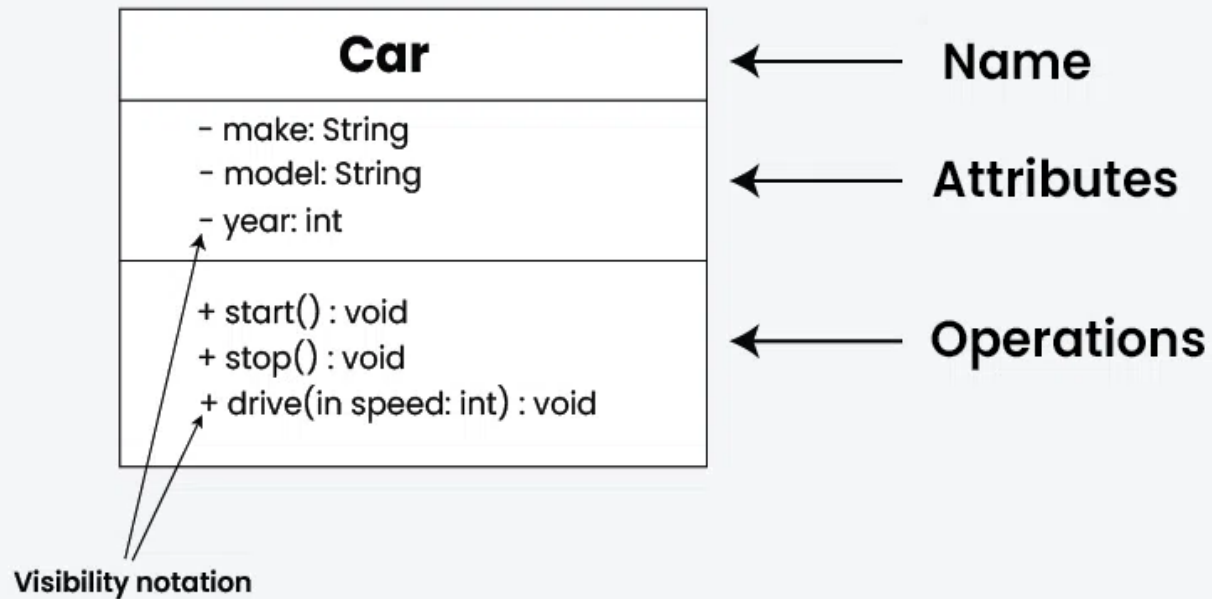
2.Middle Section:

- The middle section **constitutes the attributes**, which **describe the quality of the class**.
- The attributes have the following characteristics:
 - The attributes are written along with its visibility factors, which are **public (+)**, **private (-)**, **protected (#)**, and **package (~)**.
 - A **meaningful name should be assigned to the attribute**, which will explain its usage inside the class.

3.Lower Section:

- The lower section **contain methods or operations**.
- The **methods are represented in the form of a list**, where each **method is written in a single line**. It demonstrates **how a class interacts with data**.





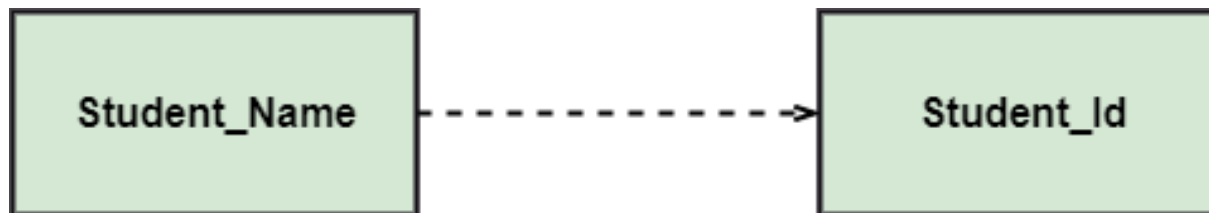
- + **Public**
- **Private**



➤ Relationships

1. Dependency:

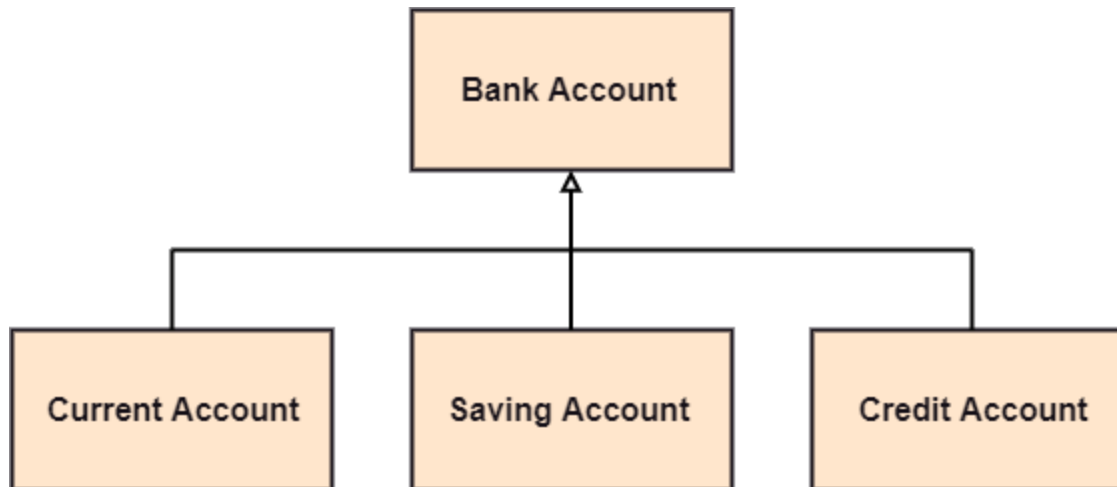
- A dependency is a **semantic relationship between two or more classes** where a **change in one class cause changes in another class**.
- It forms a **weaker relationship**.
In the following example, Student_Name is dependent on the Student_Id.



2. Generalization:

- A generalization is a relationship between a **parent class (superclass)** and a **child class (subclass)**. In this, the child class **is inherited from the parent class**.

For example, The Current Account, Saving Account, and Credit Account are the generalized form of Bank Account.



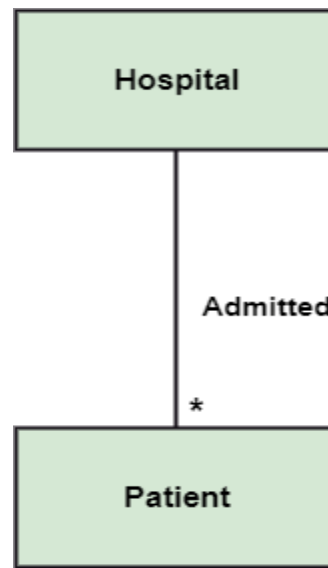
3. Association: It describes a **static or physical connection between two or more objects**. It depicts how **many objects are there in the relationship**.

For example, a department is associated with the college.



4. Multiplicity: It defines a **specific range of allowable instances of attributes**. In case if a range is not specified, one is considered as a default multiplicity.

- For example, **multiple patients are admitted to one hospital**.



5. Aggregation: An **aggregation** is a subset of association, which represents has a relationship. It is more specific than association. It defines a part-whole or part-of relationship. In this kind of relationship, **the child class can exist independently of its parent class.**

- The company encompasses a number of employees, and even if one employee resigns, the company still exists.



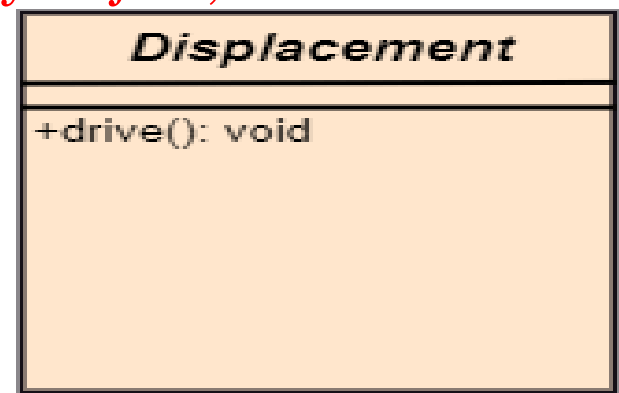
6. Composition: The **composition** is a subset of aggregation. It portrays the dependency between the parent and its child, which means **if one part is deleted, then the other part also gets discarded.** It **represents a whole-part relationship.**

- A contact book consists of multiple contacts, and if you delete the contact book, all the contacts will be lost.



7. Abstract Classes:

- In the abstract class, no objects can be a direct entity of the abstract class.
- The abstract class can neither be declared nor be instantiated.
- It is **used to find the functionalities across the classes**.
- **The notation of the abstract class is similar to that of class;**
- **The only difference is that the name of the class is written in italics.** Since it does not involve any implementation for a given function, **it is best to use the abstract class with multiple objects.**
- Let us assume that we have an **abstract class named *displacement* with a method declared inside it**, and that method will be called as a **drive ()**. Now, this abstract class method **can be implemented by any object, for example, car, bike, scooter, cycle, etc.**



UML Class Diagram for online Library Management System

Classes of Library Management System :

Library Management System class - It manages all operations of Library Management System. It is central part of organization for which software is being designed.

User Class - It manages all operations of user.

Librarian Class - It manages all operations of Librarian.

Book Class - It manages all operations of books. It is basic building block of system.

Account Class - It manages all operations of account.

Library database Class - It manages all operations of library database.

Staff Class - It manages all operations of staff.

Student Class - It manages all operations of student.



Attributes of Library Management System :

- **Library Management System Attributes** - UserType, Username, Password
- **User Attributes** - Name, Id
- **Librarian Attributes** - Name, Id, Password, SearchString
- **Book Attributes** - Title, Author, ISBN, Publication
- **Account Attributes** - no_borrowed_books, no_reserved_books, no_returned_books, no_lost_books fine_amount
- **Library database Attributes** - List_of_books
- **Staff Class Attributes** - Dept
- **Student Class Attributes** - Class

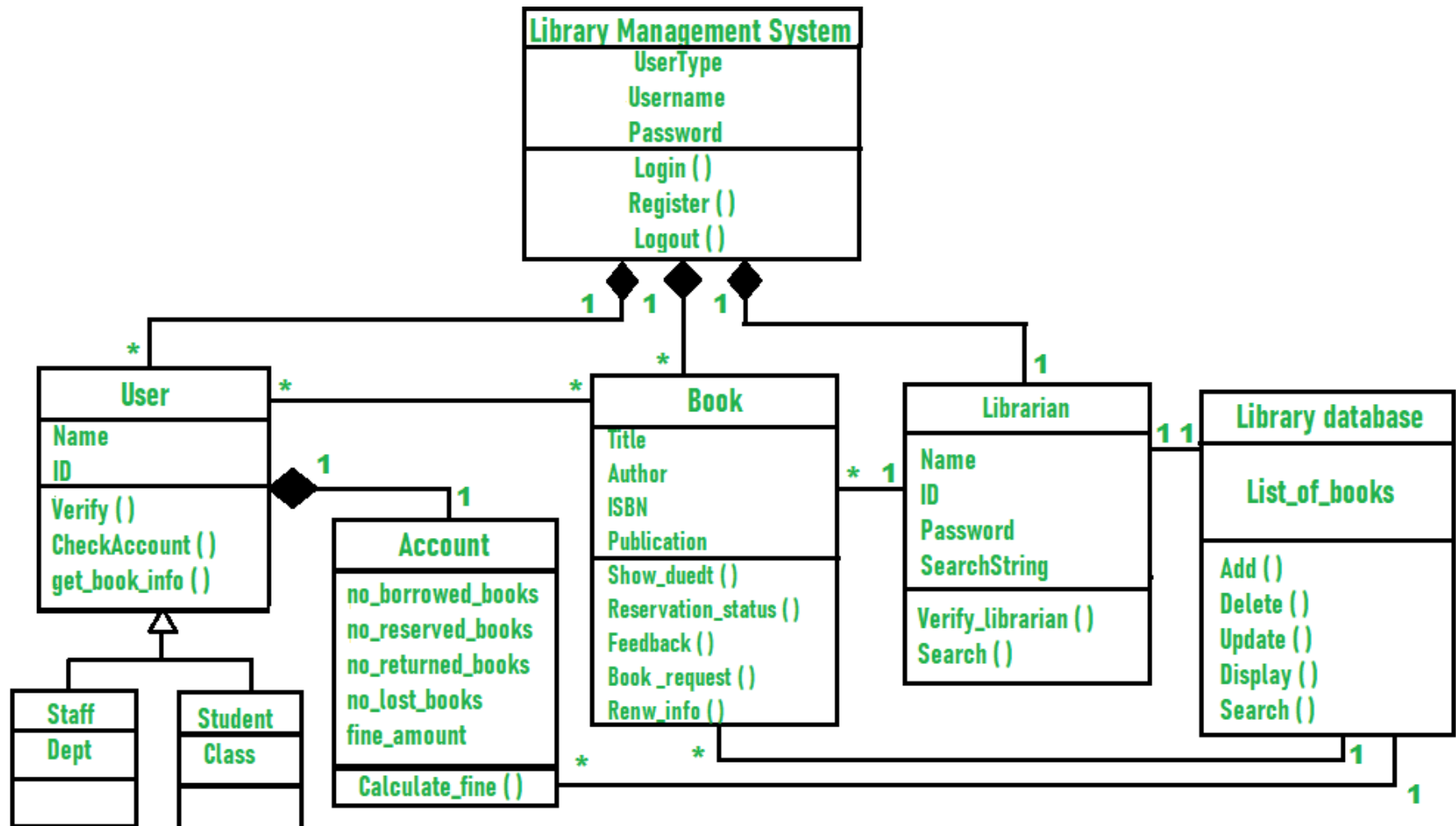


Methods of Library Management System :

- **Library Management System Methods** - Login(), Register(), Logout()
- **User Methods** - Verify(), CheckAccount(), get_book_info()
- **Librarian Methods** - Verify_librarian(), Search()
- **Book Methods** - Show_duedt(), Reservation_status(), Feedback(), Book_request(), Renew_info()
- **Account Methods** - Calculate_fine()
- **Library database Methods** - Add(), Delete(), Update(), Display(), Search()



UML Class Diagram for online Library Management System

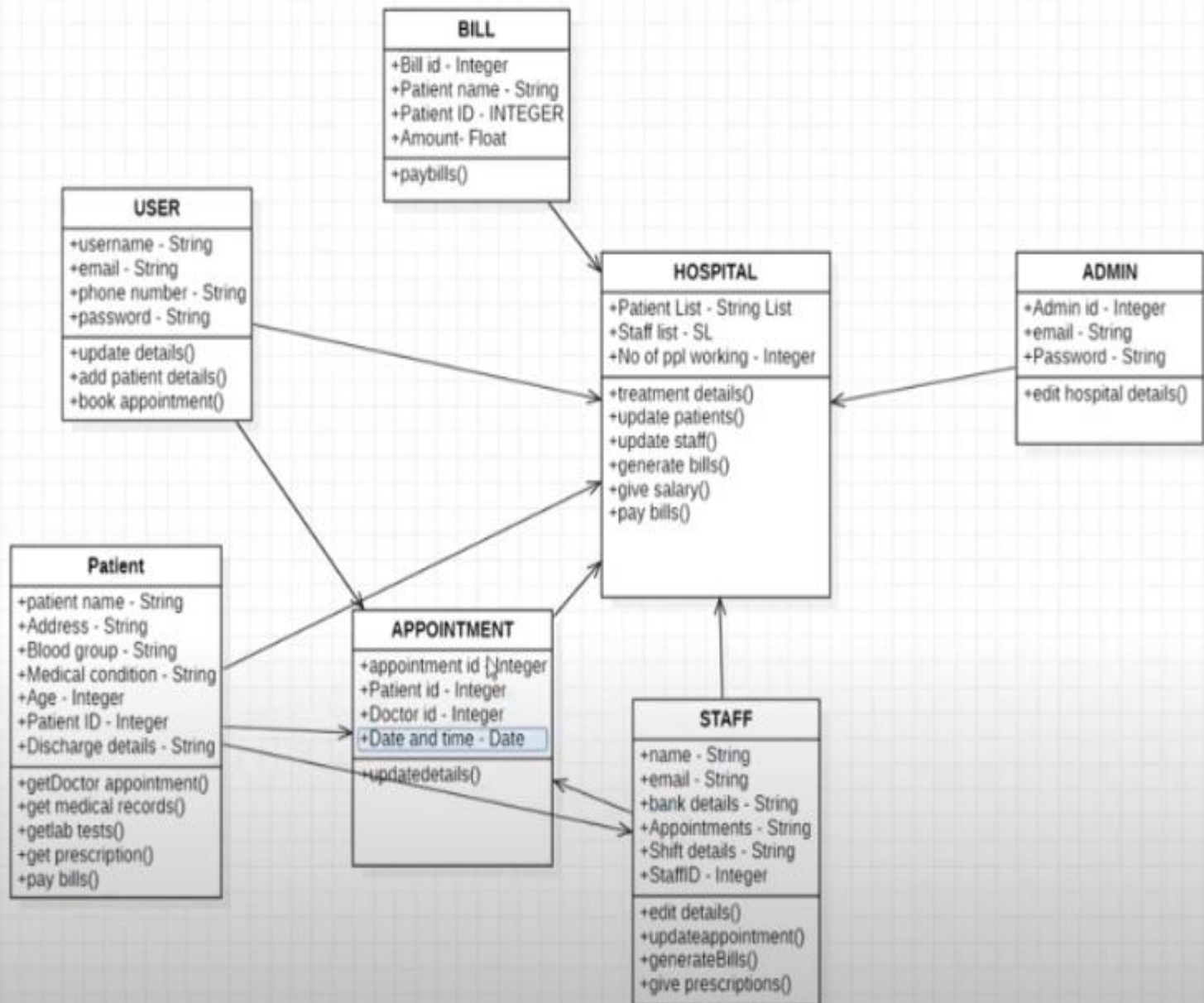


CLASS DIAGRAM FOR LIBRARY MANAGEMENT SYSTEM

UML CLASS DIAGRAM FOR ONLINE HOSPITAL MANAGEMENT SYSTEM

1. User
2. Appointment
3. Patient
4. Hospital
5. Admin
6. Bill
7. Staff



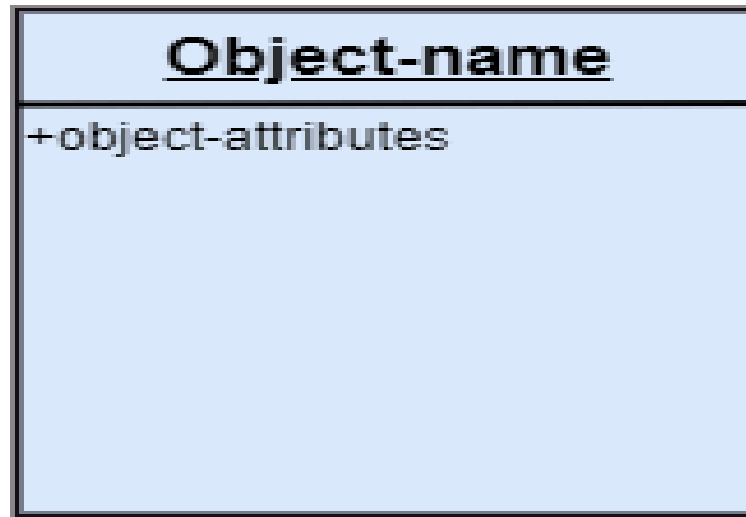


2.UML Object Diagram

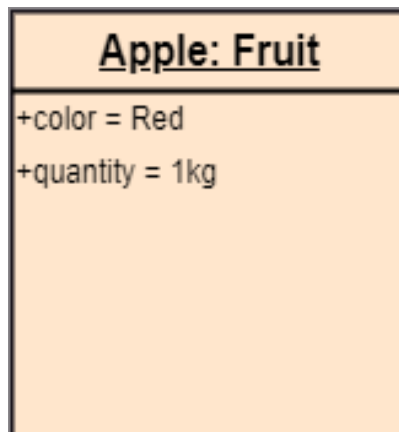
- Object diagrams are dependent on the class diagram as they are derived from the class diagram.
- It represents an instance of a class diagram.
- The objects help in portraying a static view of an object-oriented system at a specific instant.
- Both the object and class diagram are similar to some extent; the only difference is that the class diagram provides an abstract view of a system. It helps in visualizing a particular functionality of a system.



➤ Notation of an Object Diagram



➤ Example of Object Diagram



➤ Purpose of Object Diagram

- The object diagram holds the same purpose as that of a class diagram.
- The class diagram provides an abstract view which comprises of classes and their relationships, whereas the object diagram represents an instance at a particular point of time.
- It is used to understand object behavior and their relationships practically.
- It is used to get a static view of a system.
- It is used to represent an instance of a system.



➤ How to draw an Object Diagram?

- All the objects present in the system should be examined before start drawing the object diagram.
- Before creating the object diagram, the relation between the objects must be acknowledged.
- The association relationship among the entities must be cleared already.
- To represent the functionality of an object, a proper meaningful name should be assigned.



○ Class vs. Object diagram

Serial No.	Class Diagram	Object Diagram
1.	It depicts the static view of a system.	It portrays the real-time behavior of a system.
2.	Dynamic changes are not included in the class diagram.	Dynamic changes are captured in the object diagram.
3.	The data values and attributes of an instance are not involved here.	It incorporates data values and attributes of an entity.
4.	The object behavior is manipulated in the class diagram.	Objects are the instances of a class.



1 U1: user

User id =101

Password= ****

Email=123@gmail.com



2 c1: catalogue

Author name= Rajiv Chopra

Book name=DBMS

Book id= AE1478

Date= 24/11/2013



3 p1: payment

User name= 101

Method= online

sum= 370

4 N1: notification

User name= 101

email=123@gmail.com

Book detail= 5th edition



5 a1: Admin

User name= admin

Password=48*****

email=admin@gmail.com



6 I1:Inventory

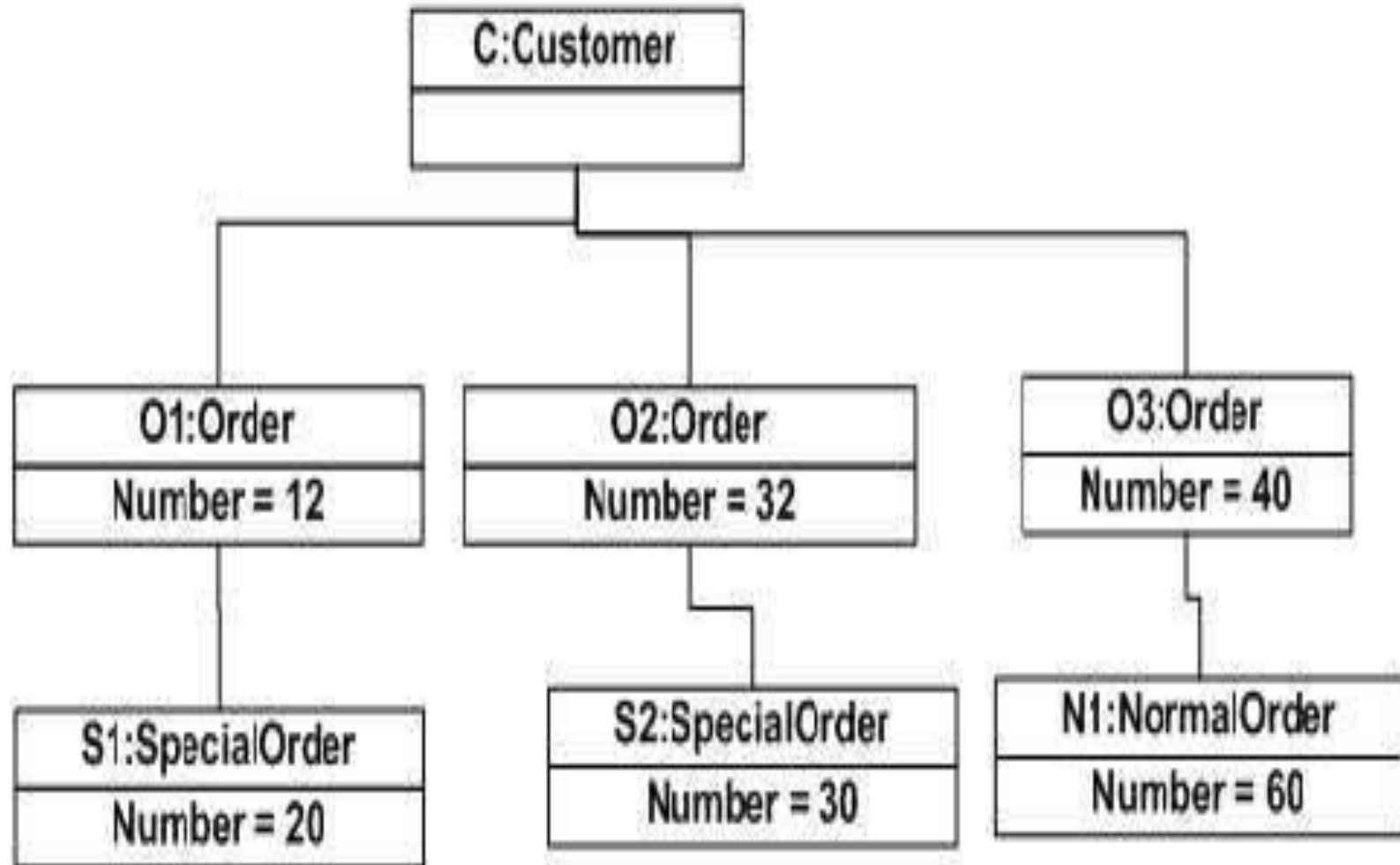
Book name= DBMS

Book id= AE1478

Author name= Rajiv Chopra

Object Diagram for Library Management System

Object diagram of an order management system



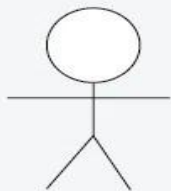
- The customer object (C) is associated with three order objects (O1, O2, and O3). These order objects are associated with special order and normal order objects (S1, S2, and N1). The customer has the following three orders with different numbers (12, 32 and 40) for the particular time considered.
- The customer can increase the number of orders in future and in that scenario the object diagram will reflect that. If order, special order, and normal order objects are observed then you will find that they have some values.
- For orders, the values are 12, 32, and 40 which implies that the objects have these values for a particular moment (here the particular time when the purchase is made is considered as the moment) when the instance is captured
- The same is true for special order and normal order objects which have number of orders as 20, 30, and 60. If a different time of purchase is considered, then these values will change accordingly.



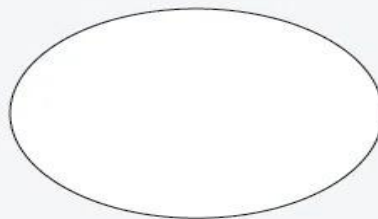
3.UML Use Case Diagram

- A Use Case Diagram is a type of Unified Modeling Language (UML) diagram that **represents the interaction between actors (users or external systems) and a system under consideration to accomplish specific goals.**
- It provides a **high-level view of the system's functionality** by illustrating the various ways users can interact with it.

Use Case Diagram Notations



Actor



UseCase



System Boundary

➤ **When to apply Use Case Diagram?**

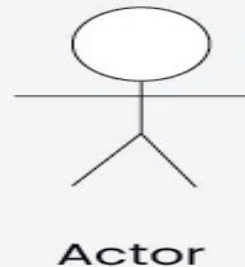
- Use case diagrams are useful in several situations.....
- When **you need to gather and clarify user requirements**, use case diagrams help visualize how different users interact with the system.
- If you're working with **diverse groups, including non-technical stakeholders**, these diagrams **provide a clear and simple way to convey system functionality**.
- During the system design phase, use case diagrams help outline user interactions and plan features, **ensuring that the design aligns with user needs**.



USE CASE DIAGRAM NOTATIONS

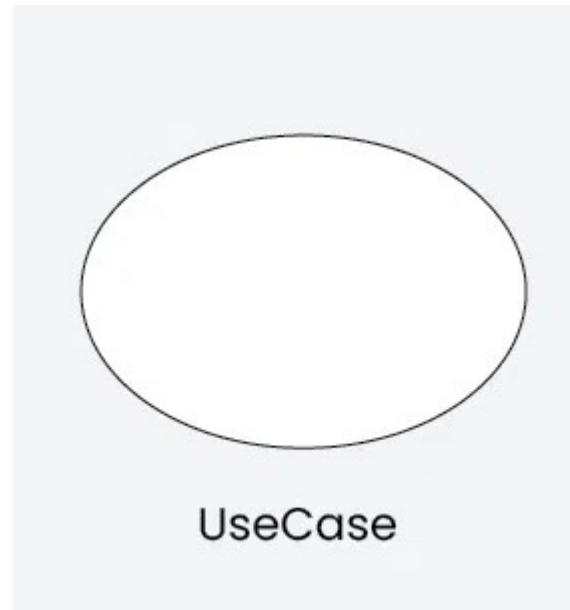
1. Actors

- Actors are external entities that interact with the system.
- These can include users, other systems, or hardware devices.
- Proper identification and understanding of actors are crucial for accurately modeling system behavior.



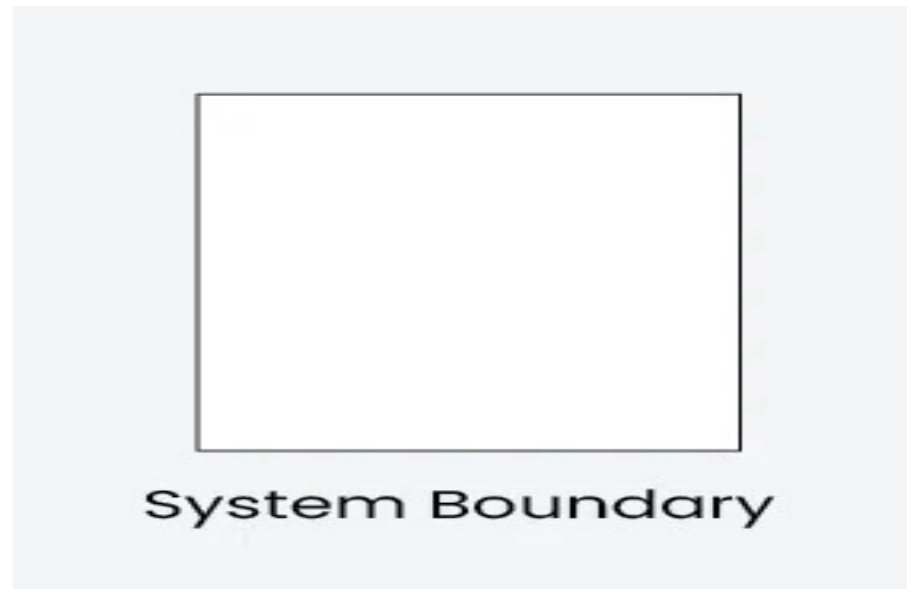
2. Use Cases

- They represent specific things your system can do.
- In the **online shopping system**, examples of use cases could be "Place Order," "Track Delivery."
- Use cases are represented by ovals.



3. System Boundary

- The system boundary is a **visual representation of the scope or limits of the system you are modeling.**
- It defines **what is inside the system and what is outside.**
- The boundary helps to **establish a clear distinction between the elements that are part of the system and those that are external to it.**
- The system boundary is **typically represented by a rectangular box** that surrounds all the use cases of the system.



HOW TO DRAW A USE CASE DIAGRAM?

1. System Analysis

- Analyze the entire system before beginning the use case diagram.
- Identify all the functionalities of the system.

2. Transform Functionalities into Use Cases

- Convert each identified functionality into a use case.
- Use these use cases in the diagram.

3. Enlist Actors

- Identify the actors that interact with the system.
- An actor can be:
 - A person, system, or private entity.
 - Any entity relevant to the system's functionalities.

4. Examine Actor-Use Case Relationships

- Define how actors interact with use cases and the system.
- Understand that an actor can interact multiple times with a use case or system during a specific instance.



USE CASE DIAGRAM FOR TICKET RESERVATION SYSTEM

1. Actor

- a. Passenger
- b. Railway agent/system admin

2. Use Cases

- a. Register
- b. Login
- c. Search for train
- d. Book tickets
- e. Payment
- f. Cancel tickets
- g. Update Detail
- h. Check status
- h. Refund payment
- i. Offer notification
- j. Statistics of tickets



USE CASE DIAGRAM FOR TICKET RESERVATION SYSTEM

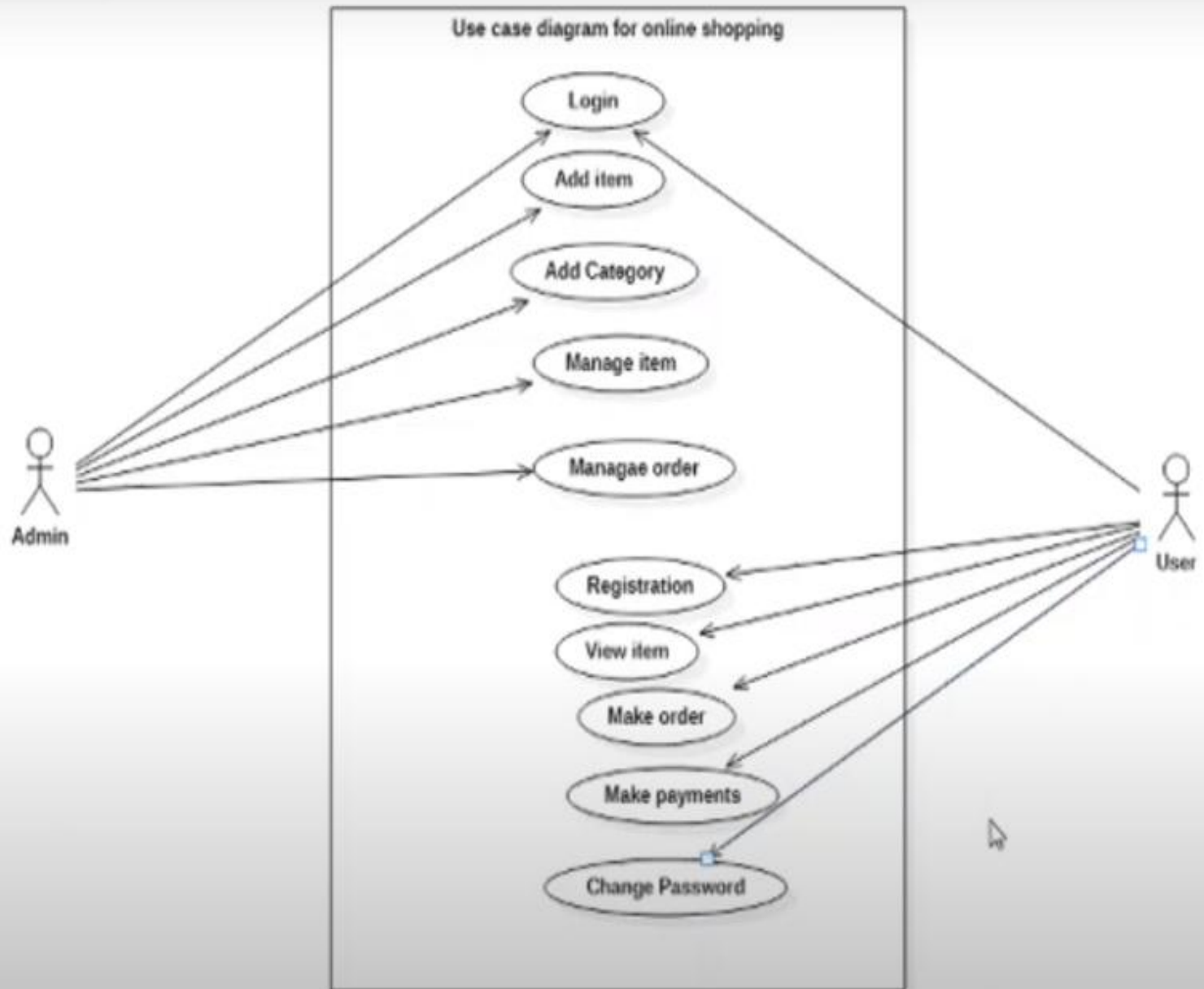


USE CASE DIAGRAM FOR ONLINE SHOPPING SYSTEM

1. Actor

- a. Customer
 - I. new customer
 - II. Registered Customer
- a. Admin



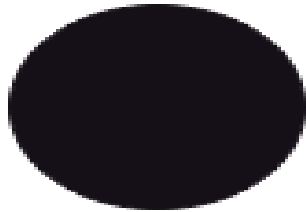


4.UML Activity Diagram

- Activity diagrams are an [essential part of the Unified Modeling Language \(UML\)](#) that help **visualize workflows, processes, or activities** within a system.
- They depict **how different actions are connected** and **how a system moves from one state to another**.
- By **offering a clear picture of both simple and complex workflows**, activity diagrams **make it easier for developers and stakeholders** to understand how various elements interact in a system.



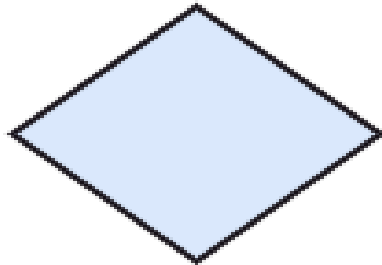
ACTIVITY DIAGRAM NOTATIONS



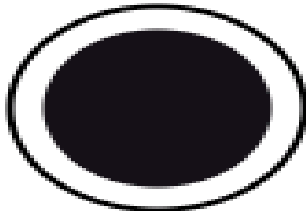
Initial State



Action box



Decision-box



Final State



1. Initial State

- The **starting state** before an activity takes place is depicted using the initial state.



Initial State

- *Here the initial state of the system **before the application is opened**.*

- Initial State symbol being used



2. Action or Activity State

- An **activity** represents execution of an **action** on objects or by objects.
- We **represent an activity using a rectangle with rounded corners**.
- Basically any **action or event that takes place** is represented using an activity.



Activity State

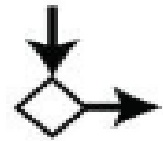
■ For example:

Consider the previous example of opening an **application**, opening the application is an activity state in the activity diagram.



3. Decision node and Branching

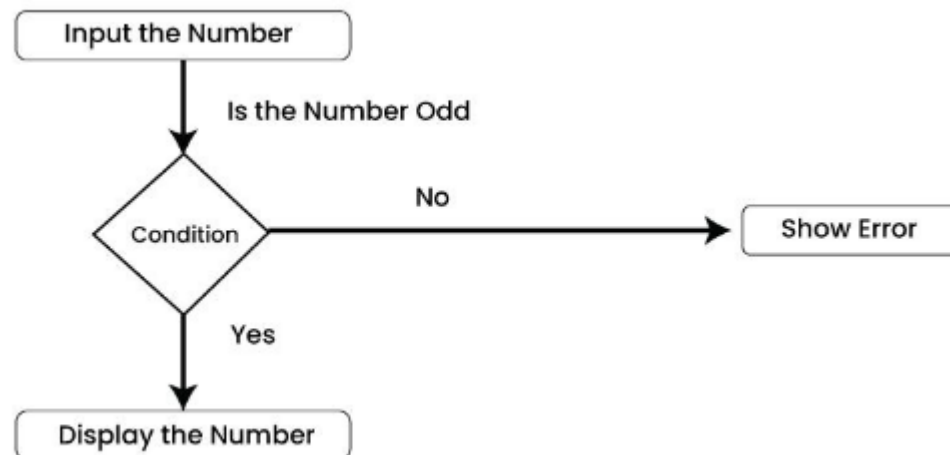
- When we need to make a decision before deciding the flow of control, we use the decision node.
- The outgoing arrows from the decision node can be labelled with conditions or guard expressions.
- It always includes two or more output arrows.



Decision node

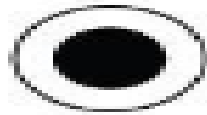
➤ For example:

- We apply the conditions on input number to display the result :
- If number is odd then display the number.
- If number is even then display the error.



11. Final State or End State

- The state which the system reaches when a particular process or activity ends is known as a Final State or End State.
- We use a filled circle within a circle notation to represent the final state in a state machine diagram.
- A system or a process can have multiple final states.



Final State



BELOW ARE THE STEPS OF HOW TO DRAW THE ACTIVITY DIAGRAM IN UML:

Step 1. Identify the Initial State and Final States:

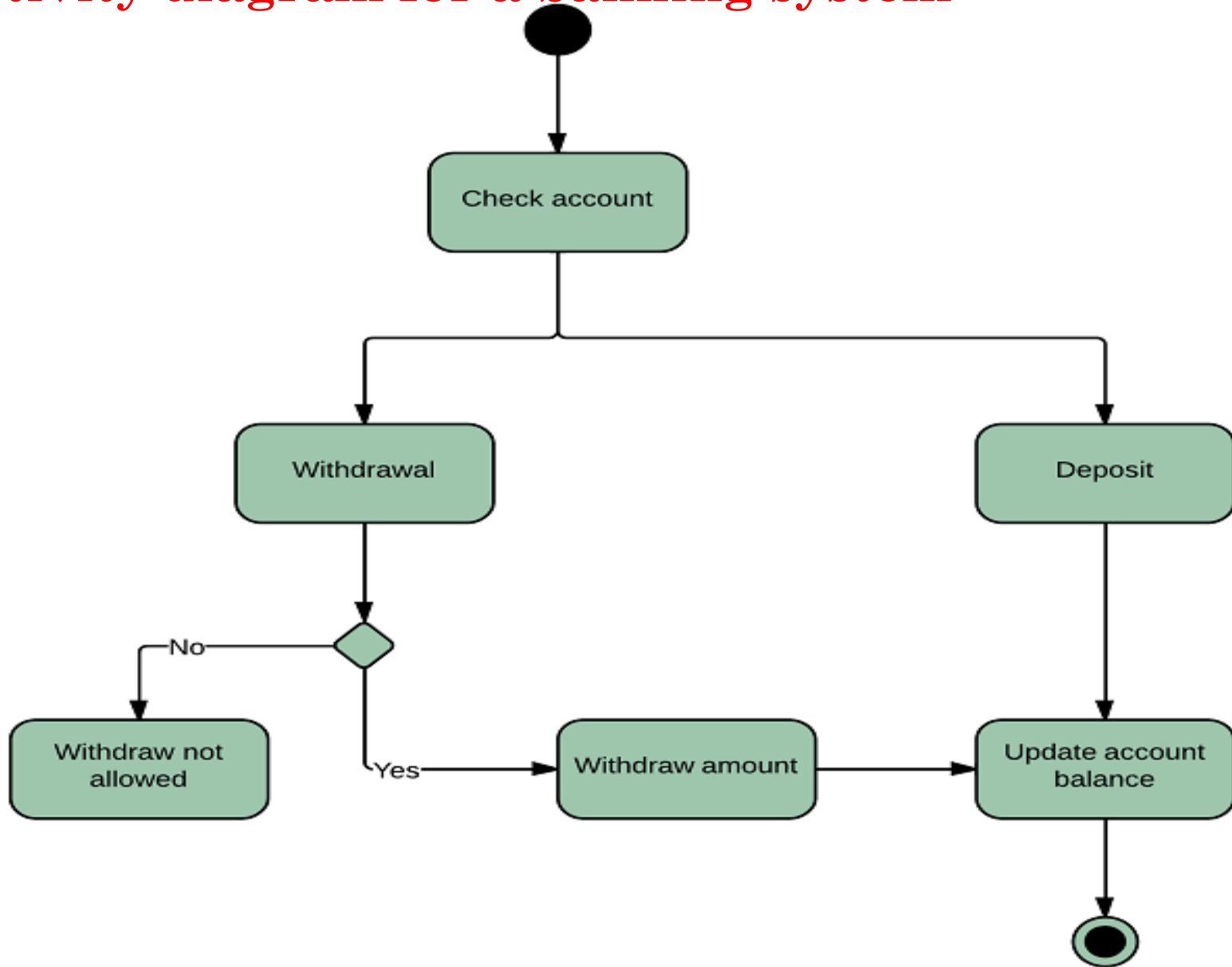
Step 2. Identify the Intermediate Activities Needed:

Step 3. Identify the Conditions or Constraints:

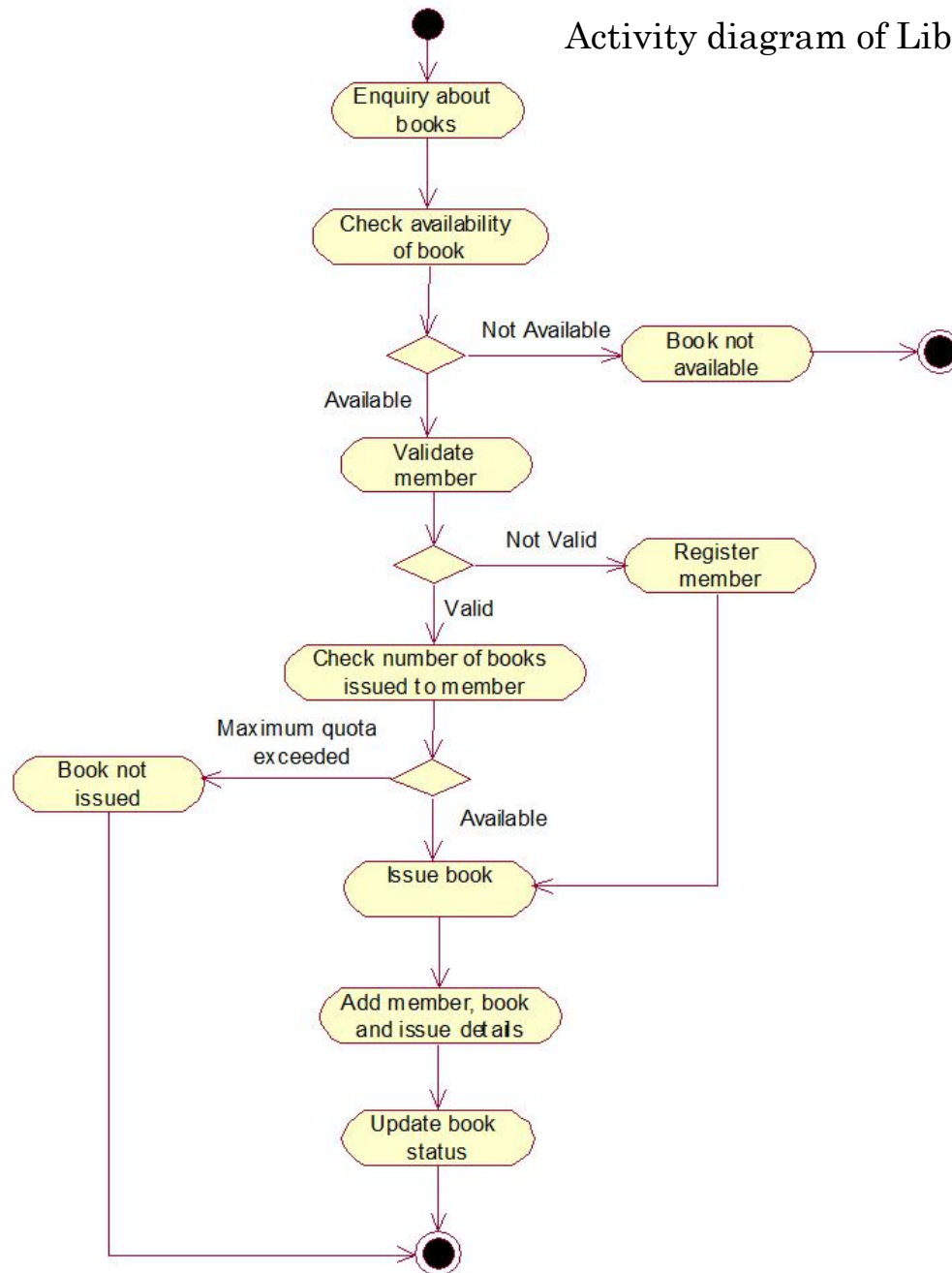
Step 4. Draw the Diagram with Appropriate Notations:



Activity diagram for a banking system



Activity diagram of Library Management System



WHAT IS A CASE STUDY?

- A **case study** is a detailed study or investigation of a particular subject, situation, organization, event, or problem over a period of time.
- It is used to understand **real-life situations**, analyze problems, and find practical solutions.



CASE STUDY: LIBRARY MANAGEMENT SYSTEM (LMS)

1. Introduction

A **Library Management System (LMS)** is a software application designed to automate and manage library operations such as book issue/return, member management, cataloging, and fine calculation. The goal is to replace manual record-keeping with a **fast, accurate, and user-friendly digital system.**

2. Problem Statement

- The traditional manual library system faces several issues:
- Time-consuming book issue and return process
- Difficulty in tracking available books
- Errors in fine calculation
- Poor record maintenance
- No real-time information for students and staff



3. Objective of the System

- Automate library operations
- Reduce human errors
- Maintain centralized book and member records
- Improve search and reporting efficiency
- Provide real-time book availability

4. Existing System (Manual)

- Records maintained in registers
- Manual search for books
- High chances of data loss
- Slow report generation
- Limited access to information



5. Proposed System (Automated LMS)

The proposed LMS is a **web/desktop-based application** that:

- Stores data in a database
- Allows quick book search
- Automates issue/return and fine calculation
- Generates reports instantly
- Provides role-based access

6. System Users

Admin / Librarian

Add/update books
Manage users
Generate reports

Students / Members

Search books
View issued books
Check due dates and fines



7. Functional Requirements

- User login & authentication
- Book management (Add, Update, Delete)
- Member management
- Book issue & return
- Fine calculation
- Search by title, author, or category
- Report generation

8. Non-Functional Requirements

High performance

Data security

User-friendly interface

Scalability

Reliability



9. Technology Stack (Example)

- **Frontend:** HTML, CSS, JavaScript
- **Backend:** Java / PHP / Python
- **Database:** MySQL / PostgreSQL
- **Tools:** XAMPP, VS Code

10. Advantages of LMS

- Saves time and manpower
- Reduces paperwork
- Accurate data handling
- Easy access to information
- Improves library efficiency



11. Conclusion

- The **Library Management System** improves overall library operations by automating routine tasks, reducing errors, and providing fast access to information.



CASE STUDY: TICKET RESERVATION SYSTEM

○ 1. Introduction

A **Ticket Reservation System (TRS)** is a software application designed to automate the process of booking, canceling, and managing tickets for transportation or events such as **railways, buses, airlines, or movies**. The system reduces manual work, improves accuracy, and provides convenience to both customers and administrators.



○ 2. Problem Statement

Traditional manual ticket booking systems face several issues:

- Long queues and waiting time
- Human errors in seat allocation
- Difficulty in maintaining records
- Limited access to booking information
- Inefficient cancellation and refund process

The organization requires an **online, centralized, and secure system** to manage ticket reservations efficiently.



3. Objectives of the System

- To allow users to book tickets online anytime
- To provide real-time seat availability
- To reduce manual effort and paperwork
- To manage cancellations and refunds easily
- To maintain accurate booking and payment records

4. Existing System

The existing system is mostly **manual or semi automated**, where:

- Tickets are booked at counters
- Records are maintained on paper or basic systems
- Seat availability is checked manually
- Reports take more time to generate



Limitations:

- Time-consuming
- Error-prone
- Poor customer experience

5. Proposed System

The proposed **Ticket Reservation System** is a **web-based application** that provides:

- Online ticket booking and cancellation
- Automatic seat allocation
- Secure payment gateway
- Instant confirmation via email/SMS
- Admin control for schedules, fares, and reports



6. System Users

User (Customer)

- Register and login
- Search routes/events
- Book and cancel tickets
- View booking history

Admin

- Manage routes/events
- Update schedules and fares
- Manage seat availability
- Generate reports



7. Functional Requirements

- User registration and authentication
- Search ticket availability
- Ticket booking and confirmation
- Online payment processing
- Ticket cancellation and refund
- Admin dashboard for management

8. Non-Functional Requirements

- High security for user data and payments
- Fast response time
- Scalability for large number of users
- User-friendly interface
- System availability 24/7



9. Technologies Used (Example)

- **Frontend:** HTML, CSS, JavaScript
- **Backend:** Java / PHP / Python / Node.js
- **Database:** MySQL / PostgreSQL
- **Payment Gateway:** Razorpay / PayPal
- **Server:** Apache / Cloud Hosting

10. Advantages of the System

- Saves time and effort
- Reduces human errors
- Improves customer satisfaction
- Easy data management and reporting
- Supports digital payments

11. Conclusion

- The **Ticket Reservation System** provides a reliable and efficient solution for managing ticket bookings in a digital environment. It enhances operational efficiency, improves user experience, and supports modern, cashless transactions.



CASE STUDY ON HOSPITAL MANAGEMENT SYSTEM

1. Introduction

- A **Hospital Management System (HMS)** is a software application designed to manage hospital operations such as patient records, appointments, billing, pharmacy, laboratory, and staff management.
- It helps hospitals improve efficiency, reduce paperwork, and provide better patient care.



2. Background of the Case

- ABC Multispeciality Hospital (Medium-sized hospital in India)
- 150 Beds
- 40 Doctors
- 60 Nurses
- 25 Administrative Staff
- 200+ Patients daily
- The hospital was using **manual record-keeping** and Excel sheets, which created many operational problems.



3. Problems Identified

- ✗ Long patient waiting time
- ✗ Misplaced patient records
- ✗ Billing errors
- ✗ Difficulty in tracking medicine stock
- ✗ Poor coordination between departments
- ✗ Manual appointment scheduling

4. Proposed Solution – Hospital Management System

- The hospital decided to implement a centralized **Hospital Management System** with the following modules:

1. Patient Management Module

Features:

- Patient registration
- Unique Patient ID generation
- Medical history storage
- Appointment scheduling



(ii) Billing & Accounts Module

- **Features:**
- Automatic bill generation
- Insurance management
- Payment tracking
- GST calculation

(iii) Pharmacy Management

- **Features:**
- Medicine stock tracking
- Expiry date alerts
- Purchase management
- Prescription integration

(iv) Laboratory Management

- **Features:**
- Test request management
- Online report generation
- Doctor access to reports
- Automated result storage



(v)Doctor & Staff Management

○ Features:

- Staff attendance
- Duty scheduling
- Salary management
- Performance tracking

5.System Architecture

- Frontend: Web Application (HTML, CSS, JavaScript)
- Backend: PHP / Java / Python
- Database: MySQL
- Cloud Server for Data Storage
- Role-Based Login System



6.Benefits After Implementation

- ✓ Reduced patient waiting time by 40%
- ✓ Accurate billing and reduced errors
- ✓ Easy access to patient history
- ✓ Better inventory control
- ✓ Improved communication between departments
- ✓ Secure and centralized database

7.Challenges Faced

- Staff training required
- Initial implementation cost
- Data migration from manual records
- System maintenance



Conclusion

- The Hospital Management System significantly improved hospital efficiency, accuracy, and patient satisfaction.
- It transformed the hospital from a manual system to a **fully digital, automated healthcare management system**, enhancing overall service quality.



CASE STUDY ON ONLINE CUSTOMER SERVICE SYSTEM

1.Introduction

- An **Online Customer Service System (OCSS)** is a web-based platform that allows customers to submit queries, complaints, feedback, and service requests digitally. It helps organizations provide fast, efficient, and trackable customer support.

2.Background of the Problem

- ABC Electronics (example company) was facing issues:
 - ✕ Delayed responses to customer complaints
 - ✕ Lost complaint records
 - ✕ No proper tracking system
 - ✕ Poor customer satisfaction
 - The company decided to develop an **Online Customer Service System** to improve service quality.
- 

3.Objectives of the System

- ✓ Provide 24/7 customer support
- ✓ Track customer complaints efficiently
- ✓ Reduce response time
- ✓ Maintain customer service history
- ✓ Improve customer satisfaction

4. System Features

Customer Registration & Login

Customers create accounts and log in securely.

Raise Ticket / Complaint

Customers submit complaints with product details and description.

Ticket Tracking

Customers can check status:

- Pending
- In Progress
- Resolved

Admin Dashboard

Admin assigns tickets to support staff.

Live Chat Support

Real-time communication between customer and support team.

Email/SMS Notifications

Automatic updates about ticket status.

Feedback System

Customers rate service quality after resolution.



5. System Architecture

- **Front-End:** HTML, CSS, JavaScript
- Back-End:** PHP / Python / Node.js
- Database:** MySQL / PostgreSQL
- Server:** Cloud / Web Server

6. Working Process

- Customer registers/login
- Customer submits complaint
- System generates Ticket ID
- Admin assigns support staff
- Staff resolves issue
- Customer receives notification
- Customer gives feedback



7.Database Tables (Example)

- Users (UserID, Name, Email, Password)
- Tickets (TicketID, UserID, Issue, Status, Date)
- Staff (StaffID, Name, Role)
- Feedback (FeedbackID, TicketID, Rating, Comments)

8.Benefits

- ✓ Faster problem resolution
- ✓ Better customer satisfaction
- ✓ Organized complaint management
- ✓ Reduced paperwork
- ✓ Improved company reputation



9.Limitations

- ⚠ Requires internet access
 - ⚠Data security concerns
 - ⚠Requires regular system maintenance

Conclusion

- The Online Customer Service System improved efficiency, reduced response time, and increased customer satisfaction for ABC Electronics. It provided a centralized, digital solution for managing customer interactions effectively.



Thank You

